מערכת מחקר **ILT** - תוכנית מעבר לפיתוח וניתוח סביבות בדיקה

מדריך יישומי למימוש אוטונומי מקבילי (*Gemini & Claude*) ותשתיות סביבת בדיקות (*Test Environment*)

1. מענה טכנולוגי: שימוש ב-Google Drive כסביבת בדיקות וחלופות מומלצות

קביעה ארכיטקטונית: תשובה חד-משמעית: לא, Google Drive אינו יכול לשמש כתשתית להקמת סביבת הבדיקות (Test Environment) של מערכת ILT.

הסבר טכני: Google Drive הוא שירות אחסון קבצים בענן (Cloud Object Storage) בעל ממשק REST API לשמירה, שליפה וניהול הרשאות של קבצים בלבד. אין בו מעבד וזיכרון (Compute Runtime) להרצת שירותי תוכנה, הוא אינו תומך בהרצת קונטיינרים (Docker), בסיסי נתונים רלציוניים מבוססי סדרות זמן (PostgreSQL / TimescaleDB), שרתי WebSockets לניהול Signaling, שרתי מדיה (go2rtc), או חיבורי WebRTC DataChannel בינאריים בזמן אמת בתדר 133Hz ו-Latency של פחות מ-15 מילי-שניות.

חלופות מומלצות להקמת סביבת הבדיקות (Test Environment)

חלופה 1 (מומלצת ביותר לפיתוח ולמעבדה): **Local Docker Compose Stack** - הרצת כלל רכיבי המערכת באמצעות Docker Compose על גבי מחשב הפיתוח המקומי. הטאבלט והלפטופ של הבוחן מתחברים ב-Wi-Fi מקומי לשרת. עלות: 0$, מהירות מירבית, אפס תלות בחיבור אינטרנט חיצוני, מושלם לבדיקות כתיבה ועט דיגיטלי.

חלופה 2 (סביבת **Staging** עננית קלה): **Managed PaaS + Serverless DB** - פריסת שרתי האפליקציה על PaaS מודרני (כגון Render או Railway) המספקים כתובת HTTPS/WSS מאובטחת, ובסיס נתונים מנוהל (PostgreSQL/TimescaleDB ב-Supabase או Neon). עלות: חינמי / דולרים בודדים בחודש. יתרון: גישה מכל מקום ללא צורך בהגדרת ראוטרים.

חלופה 3 (סביבת בדיקות ייעודית וקבועה): **Dedicated Research VPS (Hetzner / DigitalOcean / AWS Lightsail)** - מכונה וירטואלית ייעודית (Ubuntu VPS) עם Docker Compose, מנוהלת מאחורי Reverse Proxy (Nginx/Traefik) עם תעודת SSL אוטומטית. עלות: 5-10 לחודש. יתרון: סביבה זהה לחלוטין (Parity) לסביבת הייצור האקדמית.

חלופה 4 (אקוסיסטם של **Google Cloud**): **GCP Compute / Cloud Run + GCS** - במידה וקיים רצון להשתמש בתשתיות Google, החלופה המתאימה היא Google Cloud Platform (GCP) ולא Google Drive: שימוש ב-GCP Cloud Run / Compute Engine עבור השרתים, Cloud SQL עבור בסיס הנתונים, ו-Google Cloud Storage (GCS) עבור אחסון קובצי וידאו ודוחות.

2. רשימת פעולות מפורטת למעבר לשלב הפיתוח (Work Breakdown Structure)

שלב 0 - הקמת סביבת עבודה ותשתיות: הקמת סביבת Monorepo (pnpm / turborepo), הגדרת Docker Compose עבור TimescaleDB, MinIO, ו-go2rtc, והגדרת סביבת בדיקות CI/CD אוטומטית.

שלב 1 - מימוש שכבת הנתונים ו-**Core Backend API**: החלת סכמת ה-DB המלאה (כולל טבלת system_config עם ברירת מחדל של 20 שניות, 3 מדדים נפרדים, הרשאות RBAC), והרצת קובצי ה-Seed הראשוניים.

שלב 2 - מנוע תקשורת בזמן אמת ועיבוד **EMR** בינארי: הגדרת שרת Signaling מבוסס WebSockets, מימוש פרוטוקול WebRTC DataChannels (ערוץ emr-stream בלתי-אמין ב-133Hz וערוץ control-events אמין), ומימוש מפענח/אורז בינארי ברוחב 24 בתים.

שלב 3 - קיוסק התלמיד (**Tablet Student Kiosk PWA**): בניית ממשק הטאבלט ב-React/Vite Canvas, לכידת PointerEvents ברמת דיוק מיקרו-שניות, מימוש סימולטור עט (Mock Stylus) לבדיקות ללא חומרה, ומעברים חזותיים/קוליים מתוזמנים.

שלב 4 - עמדת הבוחן (**Evaluator Cockpit**): מסך צפייה ומעקב מסונכרן בזמן אמת, אפשרות השהייה/ביטול, סימון קריטריונים ידני ואוטומטי, ומודאל אישור מנהל מחקר לשינוי טיימרים.

שלב 5 - תת-מערכת אנליסט אבחון ומחקר (**Diagnostic Analyst**): פיתוח נגן Replay קינמטי אינטראקטיבי עם בקרת מהירות ומפות חום, ומנוע שליפות מחקר מבוסס DuckDB לייצוא Parquet/Arrow.

שלב 6 - מודול תוכנית ההתערבות (**Intervention Program**): מימוש שלבים 1-4, כולל חישוב אוטומטי של שורה 3 באמצעות Continuous DTW ו-Procrustes Analysis, וממשק הזנה ידני לשלב 4 הפיזי.

שלב 7 - אינטגרציה ובדיקות קבלה סופיות: בדיקות E2E מלאות ב-Playwright, בדיקות עומס ולייטנסי (<15ms), סימולציית אבחון מקצה לקצה, ואימות ייצוא נתונים.